TP de Révision – MySQL Complet (Simulation Contrôle)

Partie 1 : Création et manipulation de base et tables

Exercice 1 : Bases et tables

1. Créer une base tp_eshop_db **si elle n'existe pas**.
2. Créer la table clients avec les colonnes suivantes :
   - id INT AUTO_INCREMENT, PRIMARY KEY
   - nom CHAR(50) NOT NULL
   - prenom CHAR(40)
   - date_naissance DATE
   - adresse VARCHAR(50) DEFAULT ''
3. Ajouter une colonne email à la table clients.
4. Supprimer une colonne inutile nommée pays (si elle existe).
5. Renommer la table clients en acheteurs.

Vérifier les modifications avec DESCRIBE.

Exercice 2 : Insertion et modification de données

1. Ajouter les clients suivants dans la table acheteurs :

nom	prenom	date_naissance	adresse
Alami	Said	1980-05-22	Casablanca
Zaki	Fatima	1986-05-22	Agadir
Fadani	Ali	1990-08-10	Casablanca

Formateur : AARAB HAFID

2. Modifier le client avec id=1 pour changer son nom en Fadani et son adresse en Rabat.
3. Supprimer le client avec id=3.
4. Sélectionner tous les clients de Casablanca triés par nom.

Partie 2 : Vues et tables temporaires

1. Créer une **vue** vw_clients_agadir contenant tous les clients dont l'adresse est Agadir (insensible à la casse).
2. Créer une **table temporaire** temp_clients avec la structure de acheteurs mais sans données.
3. Supprimer la table temporaire.

Partie 3 :– Procédures et Fonctions

Exercice 3 : Procédures stockées

1. Créer une procédure sp_client_par_ville avec un paramètre d'entrée ville qui retourne tous les clients de cette ville (insensible à la casse).
2. Appeler la procédure pour la ville 'Agadir'.

Exercice 4 : Fonctions

1. Créer une fonction uf_nombre_clients_par_ville(ville VARCHAR(50)) qui retourne le nombre de clients dans une ville.
2. Tester la fonction pour la ville 'Casablanca'.

Partie 4 : Triggers

1. Ajouter une colonne date_modification à la table acheteurs.

2. Créer un trigger maj_date_modif qui met automatiquement date_modification à la date et heure actuelle **avant chaque mise à jour** sur acheteurs.
3. Vérifier le trigger en modifiant un client.

Partie 5 : Optimisation SQL

1. Créer la table commandes avec :
   - id INT AUTO_INCREMENT PRIMARY KEY
   - client_id INT
   - date_commande DATE
   - montant DECIMAL(10,2)
2. Insérer au moins **10 commandes** réparties sur les clients existants.
3. Écrire une requête pour **trouver toutes les commandes des clients d'Agadir** en utilisant **EXISTS** au lieu de IN.
4. Créer un **index** sur la colonne client_id dans commandes.
5. Écrire une requête avec **INNER JOIN** pour lister nom, prenom et date_commande pour toutes les commandes.
6. Utiliser **EXPLAIN** pour analyser la requête et indiquer si l'index est utilisé.

Partie 6 : Sécurité MySQL

Exercice 6 : Gestion des utilisateurs et privilèges

1. Créer un utilisateur ali sur localhost avec un mot de passe sécurisé.
2. Accorder uniquement les privilèges SELECT et INSERT sur tp_eshop_db.acheteurs.

3. Vérifier les privilèges de l'utilisateur.

4. Modifier le mot de passe de ali.
5. Créer un rôle role_consultation avec le privilège SELECT sur tp_eshop_db.*.
6. Assigner le rôle à l'utilisateur ali.

Exercice 7 : Prévention injection SQL

1. Créer une requête préparée pour sélectionner un client par id.
2. Exécuter la requête préparée avec id=2.

Partie 7 : Questions de réflexion

1. Pourquoi ne faut-il pas utiliser SELECT * sur de grandes tables ?
2. Quelle est la différence entre une **procédure** et une **fonction** en MySQL ?
3. Qu'est-ce qu'un **trigger** et dans quel cas serait-il utile ?
4. Pourquoi utiliser un **index** sur une colonne fréquemment utilisée dans WHERE ou JOIN ?
5. Explique la différence entre **PRIMARY KEY** et **UNIQUE**.
6. Que se passe-t-il si on oublie le WHERE dans un UPDATE ou DELETE ?
7. Pourquoi les colonnes de type texte très long ne sont pas recommandées pour un index ?
8. Explique la différence entre **IN**, **OUT** et **INOUT** dans les procédures.
9. Quelles sont les différences entre **INNER JOIN** et **LEFT JOIN** ?
10. Pourquoi l'usage de fonctions comme UPPER() ou LOWER() dans le WHERE peut ralentir une requête ?
11. Quand utiliser EXISTS plutôt que IN ?
12. Quelle est la différence entre UNION et UNION ALL ?
13. Pourquoi est-il important d'utiliser des **requêtes préparées** pour les données entrées par les utilisateurs ?
14. Quelle est la différence entre **table temporaire** et table normale ?
15. Pourquoi limiter les privilèges d'un utilisateur à ce qui est strictement nécessaire (principe du "least privilege") ?
16. Pourquoi ne faut-il pas permettre à root de se connecter à distance ?

17. Quelles sont les différences entre **cache** et **buffer** dans MySQL et comment cela impacte la performance ?
18. Explique la différence entre les moteurs **InnoDB** et **MyISAM**.
19. Pourquoi utiliser EXPLAIN pour analyser une requête SQL ?

20. Que signifie DETERMINISTIC dans la création d'une fonction MySQL ?

21. Qu'est-ce qu'une **vue** et quels sont ses avantages par rapport à une requête directe ?
22. Quels sont les avantages et les inconvénients de l'utilisation des **triggers** dans une base de données ?
23. Pourquoi utiliser des **tables temporaires** plutôt que créer une table normale pour des calculs intermédiaires ?
24. Pourquoi % au début d'un LIKE ralentit la requête ?
25. Explique comment un rôle peut simplifier la gestion des utilisateurs dans MySQL.
26. Que fait la clause READS SQL DATA dans une fonction MySQL ?
27. Quelle est la différence entre un **cache de requêtes** et un **index** ?
28. Explique le concept de **N+1 problem** et comment l'éviter.
29. Pourquoi les sauvegardes doivent-elles être sécurisées et protégées ?
30. Quelles sont les bonnes pratiques pour sécuriser un fichier de configuration MySQL (my.cnf ou my.ini) ?
31. Comment SSL protège-t-il les connexions MySQL ?
32. Pourquoi les requêtes DISTINCT et ORDER BY peuvent ralentir une grande table ?
33. Quelle est la différence entre CREATE USER et GRANT ?
34. Pourquoi utiliser des **variables locales** dans une procédure ou fonction MySQL ?
35. Quelles sont les implications d'une table avec **beaucoup de colonnes modifiées fréquemment** sur la performance si un index est présent ?